

Apache Web Server – Virtual Hosting

Questions :

- i. Install Apache Web Server
- ii. Create an index.html page in default document root
- iii. Access the website using mysamplewebsite.itfs

Note: Create a custom Virtual Hosting configuration File

Study in detail about Apache default files, document root and modules

Task i – Install Apache Web Server

Question

Install Apache Web Server.

Commands Used

```
sudo dnf install httpd -y
sudo systemctl enable --now httpd
sudo systemctl status httpd
```

Procedure

1. Installed the Apache HTTP Server package using dnf.
2. Enabled and started the Apache service.
3. Checked the status of the Apache service.

Output

```
[ec2-user@ip-172-31-4-243 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 05:29:47 UTC; 1h 10min ago
     Invocation: c6a382cebc944a25a51ac8973b6ace3d
       Docs: man:httpd.service(8)
   Process: 7141 ExecReload=/usr/sbin/httpd $OPTIONS -k graceful (code=exited, status=0/SUCCESS)
    Main PID: 6105 (httpd)
   Status: "Total requests: 22; Idle/Busy workers 100/0; Requests/sec: 0.00522; Bytes served/sec: 4 B/sec"
     Tasks: 230 (limit: 4156)
    Memory: 18.1M (peak: 20.3M)
       CPU: 4.456s
    CGroup: /system.slice/httpd.service
            └─6105 /usr/sbin/httpd -DFOREGROUND
              └─7143 /usr/sbin/httpd -DFOREGROUND
                └─7144 /usr/sbin/httpd -DFOREGROUND
                  └─7174 /usr/sbin/httpd -DFOREGROUND
                    └─7175 /usr/sbin/httpd -DFOREGROUND
                      └─7322 /usr/sbin/httpd -DFOREGROUND
```

Result

Apache Web Server was successfully installed and started.

Task ii – Create an index.html Page in Default Document Root

Question

Create an index.html page in the default document root.

Commands Used

```
sudo vi /var/www/html/index.html
```

Procedure

1. Opened the default Apache document root using vi.
2. Created/edited the index.html file.
3. Added the custom HTML content.
4. Saved and exited the file.

Output

```
ec2-user@ip-172-31-4-243:~  
|html>  
<h1>  
This is Mohammed Sanjid  
</h1?>  
This is a sample web hosting  
</html>
```

Result

The custom `index.html` page was successfully created in the default Apache document root `/var/www/html`.

Task iii – Access the Website Using `mysamplewebsite.itfs`

Question

Access the website using `mysamplewebsite.itfs`.

Commands Used

No Linux command required.

Procedure

1. Opened the Windows hosts file as Administrator.
2. Added the EC2 instance IP address with the domain name: `13.234.31.37
mysamplewebsite.itfs`
3. Saved the hosts file.
4. Opened the following address in the browser:

<http://mysamplewebsite.itfs>

5. Verified that the website was displayed.

Output



Result

The website was successfully accessed using the domain name `mysamplewebsite.itfs`.

Task 4 – Create Custom Virtual Hosting Configuration

Question

Create a custom Virtual Hosting configuration file.

Commands Used

Create the second website directory:

```
sudo mkdir -p /var/www/html1
```

Create the second website page:

```
sudo vi /var/www/html1/index1.html
```

Create the Virtual Host configuration files:

```
sudo vi /etc/httpd/conf.d/sanjid.conf
sudo vi /etc/httpd/conf.d/sanjid1.conf
```

Reload Apache:

```
sudo systemctl reload httpd
```

Procedure

1. Used the existing default Apache document root `/var/www/html` for the first website.
2. Created a separate directory `/var/www/html1` for the second website.
3. Created the `index1.html` page inside `/var/www/html1`.
4. Created the custom Virtual Host configuration file `sanjid.conf`.
5. Configured `sanjid.com` to use `/var/www/html` as its document root.
6. Created the second Virtual Host configuration file `sanjid1.conf`.
7. Configured `sanjid1.com` to use `/var/www/html1` as its document root.
8. Reloaded Apache to apply the Virtual Host configurations.
9. Accessed both websites using their respective domain names.

Inputs

Website 1:

Domain: `sanjid.com`
Document Root: `/var/www/html`

Website 2:

Domain: `sanjid1.com`
Document Root: `/var/www/html1`

Second website file:

`/var/www/html1/index1.html`

Virtual Host configuration files:

`/etc/httpd/conf.d/sanjid.conf`
`/etc/httpd/conf.d/sanjid1.conf`

Output



This is Mohammed Sanjid

This is a sample web hosting



This is second one to test virtual hosting

Result

Successfully configured **name-based Virtual Hosting** using two custom Apache configuration files, `sanjid.conf` and `sanjid1.conf`, to host two websites using separate document roots.

Study: Apache Default Files, Document Root and Modules

1. Apache Default Files

Apache stores its main configuration files and website files in specific locations. On **RHEL**, the important Apache files and directories are:

File / Directory	Purpose
/etc/httpd/conf/httpd.conf	Main Apache configuration file
/etc/httpd/conf.d/	Directory for additional Apache configuration files
/etc/httpd/conf.modules.d/	Contains configuration files for Apache modules
/var/www/html/	Default Apache document root
/var/log/httpd/	Contains Apache log files
/var/log/httpd/access_log	Records client requests
/var/log/httpd/error_log	Records Apache errors and warnings
/usr/sbin/httpd	Apache server executable
/usr/lib64/httpd/modules/	Contains Apache module files

Main Configuration File

The main Apache configuration file is:

```
/etc/httpd/conf/httpd.conf
```

It contains important settings such as the server configuration, listening port, document root, directory permissions, and other Apache directives.

Additional configuration files can be placed inside:

```
/etc/httpd/conf.d/
```

In this practical, the custom Virtual Host configuration files were:

```
/etc/httpd/conf.d/sanjid.conf  
/etc/httpd/conf.d/sanjid1.conf
```

2. Apache Document Root

The **document root** is the directory from which Apache serves website files to clients.

The default document root on RHEL is:

```
/var/www/html/
```

For example:

```
/var/www/html/index.html
```

When a user accesses a website, Apache searches the configured document root for the requested website files.

In this practical:

```
sanjid.com → /var/www/html
```

A separate document root was created for the second website:

```
sanjid1.com → /var/www/html1
```

This allows multiple websites to be hosted on the same Apache server using different document roots.

3. Apache Modules

Apache modules are components that extend the functionality of the Apache Web Server.

Modules provide features such as:

- SSL/HTTPS support
- URL rewriting
- Authentication
- Proxy functionality
- Directory handling
- Security features
- Dynamic content support

Some commonly used Apache modules include:

Module	Purpose
mod_ssl	Provides SSL/TLS and HTTPS support
mod_rewrite	Rewrites and redirects URLs
mod_proxy	Provides proxy functionality
mod_headers	Allows modification of HTTP headers
mod_autoindex	Generates directory listings
mod_auth_basic	Provides basic authentication

Apache modules are generally located in:

```
/usr/lib64/httpd/modules/
```

Module configuration files can be found under:

```
/etc/httpd/conf.modules.d/
```

Importance of Modules

Modules make Apache flexible because additional functionality can be enabled without changing the basic Apache server. They allow Apache to support requirements such as **HTTPS, authentication, URL rewriting, proxying, and security**.

Conclusion

Apache uses a structured configuration and file system. The main configuration is stored in `/etc/httpd/conf/httpd.conf`, additional configurations such as Virtual Hosts can be stored in `/etc/httpd/conf.d/`, website files are served from the configured **DocumentRoot**, and **modules** extend Apache's functionality.